SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV
COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

A. Process live video streams in real time (<30 FPS latency)
B. Work under varying lighting conditions
C. Use limited memory and CPU resources
D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

ANS:

Real-Time Face Recognition System (Edge Deployment)

Problem Analysis

The objective is to design a real-time face recognition system for a low-power edge device such as Raspberry Pi. The system should process live video with low latency, work under different lighting conditions, use limited CPU and memory, and recognize faces from a predefined dataset.

End-to-End Computer Vision Pipeline

1. Image Acquisition

A USB camera or Raspberry Pi camera is used to capture live video frames.

```
cap = cv2.VideoCapture(0)
```

The camera continuously provides frames to the system for processing.

2. Preprocessing

The captured frame is resized to a smaller resolution such as 320 × 240 pixels to reduce computation.

```
frame = cv2.resize(frame, (320, 240))
```

The image is then converted from BGR to grayscale.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Histogram equalization is applied to improve image contrast under varying lighting conditions.

```
gray = cv2.equalizeHist(gray)
```

This stage reduces memory usage and improves robustness to illumination changes.

3. Face Detection

The Haar Cascade classifier available in OpenCV is used to detect faces.

```
face_cascade = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml"
)

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5
)
```

Haar Cascade is selected because it is lightweight and computationally efficient for low-power devices such as Raspberry Pi.

4. Feature Extraction

For every detected face, the face region or Region of Interest (ROI) is extracted.

```
faceROI = gray[y:y+h, x:x+w]
```

Local Binary Pattern (LBP) features are used to represent facial texture. LBP is computationally simple and provides reasonable robustness to lighting variations.

5. Face Recognition

The extracted facial features are compared with the predefined face dataset using the LBPH (Local Binary Pattern Histogram) face recognizer.

```
recognizer = cv2.face.LBPHFaceRecognizer_create()
```

```
label, confidence = recognizer.predict(faceROI)
```

If the confidence value is within the selected threshold, the corresponding person's name is displayed. Otherwise, the face is classified as "Unknown".

```
if confidence < threshold:
    print("Recognized")
else:
    print("Unknown")
```

6. Output

The recognized person's name and a bounding box are displayed on the live video.

```
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.putText(frame, name, (x, y-10),
        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
```

The final output is a live video containing the detected face and the person's identity.

Optimization for Edge Deployment

The following techniques are used to satisfy the limited CPU and memory constraints:

1. Resize frames to a lower resolution such as 320 × 240.
2. Convert images to grayscale to reduce memory requirements.
3. Use lightweight Haar Cascade face detection.
4. Use LBPH instead of computationally expensive deep-learning models.
5. Skip some frames when necessary to reduce CPU usage.
6. Track detected faces between detection frames instead of detecting faces in every frame.
7. Use multithreading to separate camera capture and image processing.

Constraint Satisfaction

Constraint	Solution
Real-time processing	Image resizing, Haar Cascade and LBPH provide fast processing
Varying lighting	Histogram equalization and LBP improve illumination robustness
Limited CPU	Lightweight OpenCV algorithms are used

Constraint	Solution
Limited memory	Grayscale and low-resolution frames reduce memory usage
Predefined dataset	LBPH recognizer compares detected faces with trained faces
Unknown faces	Confidence threshold classifies unmatched faces as Unknown

Complete Pipeline

Camera Input

↓

Frame Acquisition

↓

Image Resizing

↓

Grayscale Conversion

↓

Histogram Equalization

↓

Haar Cascade Face Detection

↓

Face ROI Extraction

↓

LBP Feature Extraction

↓

LBPH Face Recognition

↓

Compare with Predefined Dataset

↓

Recognized Person / Unknown

↓

Display Final Output

Conclusion

The proposed OpenCV-based face recognition system uses a lightweight pipeline consisting of camera acquisition, resizing, grayscale conversion, histogram equalization, Haar Cascade face detection, LBP feature extraction, and LBPH recognition. The design is suitable for Raspberry Pi because it requires relatively low CPU and memory resources while providing

real-time face detection and recognition. Frame skipping and face tracking can further improve performance and help maintain the required real-time processing speed.

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

A. Detects motion and unusual activities
B. Generates alerts in real time
C. Works with standard CCTV video feeds
D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

ANS:

Automated Video Surveillance with Alert System

Problem Analysis

The objective is to design an automated video surveillance system using OpenCV that can detect motion and unusual activities from standard CCTV feeds. The system should generate real-time alerts while minimizing false positives caused by noise, moving trees, changing lighting, and other dynamic background conditions.

End-to-End Computer Vision Pipeline

1. Video Acquisition

The CCTV camera feed is captured using OpenCV. The system can process live camera feeds or recorded video.

```
cap = cv2.VideoCapture("cctv_video.mp4")
```

For a live camera, 0 can be used instead of the video file name.

```
cap = cv2.VideoCapture(0)
```

2. Preprocessing

Each video frame is resized to a smaller resolution to reduce processing time.

```
frame = cv2.resize(frame, (640, 480))
```

The frame is converted to grayscale because motion detection does not require color information.

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

Gaussian blur is applied to reduce camera noise and small unwanted movements.

gray = cv2.GaussianBlur(gray, (21, 21), 0)

3. Motion Detection

The previous frame is compared with the current frame to identify changes.

diff = cv2.absdiff(previous_frame, gray)

A threshold is applied to separate moving objects from the background.

_, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

Morphological operations such as dilation are used to remove small gaps and connect nearby motion regions.

thresh = cv2.dilate(thresh, None, iterations=2)

4. Feature Detection and Object Identification

Contours are detected from the threshold image.

```
contours, _ = cv2.findContours(
    thresh,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

Very small contours are ignored because they are usually caused by noise.

```
for contour in contours:
    if cv2.contourArea(contour) < 500:
        continue
```

A bounding box is created around significant moving objects.

```
x, y, w, h = cv2.boundingRect(contour)
cv2.rectangle(frame, (x, y), (x+w, y+h), (0,255,0), 2)
```

5. Unusual Activity Detection

The system can identify suspicious activity using conditions such as:

- Large or sudden movement.
- Movement in restricted areas.
- A person remaining in a restricted area for a long time.
- Movement during predefined inactive hours.
- Multiple objects entering a restricted region.

For example, a Region of Interest (ROI) can be defined as a restricted area. If motion is detected continuously inside this area, the system considers it unusual activity.

6. False Positive Reduction

To minimize false alerts, the system uses several techniques:

1. **Minimum contour area:** Small movements caused by noise are ignored.
2. **Gaussian blur:** Reduces sensor and video noise.
3. **Frame persistence:** An alert is generated only when motion continues for several consecutive frames.
4. **Region of Interest:** Motion outside important areas can be ignored.
5. **Background updating:** The background model is periodically updated to handle gradual environmental changes.
6. **Object tracking:** Detected objects can be tracked across multiple frames to distinguish real movement from temporary noise.

7. Dynamic Background Handling

Dynamic backgrounds such as trees, shadows, rain, and changing illumination can produce false motion.

To handle this, OpenCV background subtraction methods such as MOG2 can be used.

```
backSub = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=50,
    detectShadows=True
)

fgMask = backSub.apply(frame)
```

MOG2 continuously adapts its background model, making it more suitable for CCTV environments where the background can change.

8. Real-Time Alert Generation

When significant motion or suspicious activity is detected for a specified number of consecutive frames, an alert is generated.

```
if motion_detected:
    print("ALERT: Unusual activity detected!")
```

The alert can be extended to:

- Sound an alarm.

- Display a warning message.
- Save the current frame.
- Record the video segment.
- Send a notification to a security system.

9. Output

The final output contains the CCTV video with bounding boxes around detected moving objects and an alert message when suspicious activity is detected.

Example:

CCTV Video

↓

Motion Detected

↓

Object Tracking

↓

Unusual Activity?

↓

YES → Generate Alert

↓

Save Frame / Record Video / Notify Security

Complete Pipeline

CCTV Video Feed

↓

Frame Acquisition

↓

Frame Resizing

↓

Grayscale Conversion

↓

Gaussian Blur

↓

Background Subtraction / Frame Difference

↓

Thresholding

↓

Morphological Filtering

↓

Contour Detection

↓

Object/Feature Analysis

↓

Motion Tracking

↓

Unusual Activity Detection

↓

Alert Generation

↓

Display / Record / Notify

Handling the Given Constraints

Requirement	**Solution**
Motion detection	Frame differencing and background subtraction
Unusual activity	ROI analysis, object tracking and persistence checking
Real-time alerts	Process frames continuously and trigger alerts immediately
Standard CCTV feeds	OpenCV VideoCapture supports camera and video feeds
Noise reduction	Gaussian blur and minimum contour-area filtering
Dynamic backgrounds	MOG2 adaptive background subtraction
False positive reduction	ROI, persistence checks, tracking and noise filtering
Limited processing time	Resizing and lightweight OpenCV operations

Conclusion

The proposed OpenCV-based surveillance system uses video acquisition, preprocessing, background subtraction, thresholding, contour detection, object tracking, and activity analysis to detect suspicious motion. Gaussian filtering removes noise, while MOG2 adapts to dynamic backgrounds. Minimum-area filtering, ROI restrictions, and consecutive-frame verification reduce false positives. The lightweight processing pipeline allows real-time operation with standard CCTV feeds and generates alerts whenever unusual activity is detected.

3. OCR System for Low-Quality Documents

A digitization company needs an **OCR system** that:

A. Extracts text from low-resolution, noisy scanned images
B. Handles varying fonts and illumination
C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

ANS:

OCR System for Low-Quality Documents

Problem Analysis

The objective is to design an OCR (Optical Character Recognition) system using OpenCV to extract text from low-resolution and noisy scanned documents. The system should handle different fonts and illumination conditions and process a large number of documents efficiently.

End-to-End Image Processing Pipeline

1. Image Acquisition

The scanned document is loaded using OpenCV.

```
import cv2

image = cv2.imread("document.jpg")
```

The system can process multiple scanned images sequentially for batch processing.

2. Image Resizing

Low-resolution images are enlarged to improve character visibility.

```
image = cv2.resize(
    image,
    None,
    fx=2,
    fy=2,
    interpolation=cv2.INTER_CUBIC
)
```

Upscaling makes small characters clearer and improves OCR recognition.

3. Grayscale Conversion

The image is converted from color to grayscale.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Grayscale reduces the image from three color channels to one channel, reducing memory and processing requirements.

4. Noise Removal

Scanned documents may contain dust, scratches, and scanning noise. Gaussian or median filtering can be used to remove this noise.

```
gray = cv2.GaussianBlur(gray, (5, 5), 0)
```

For salt-and-pepper noise, median filtering can be used:

```
gray = cv2.medianBlur(gray, 3)
```

Noise removal prevents unwanted pixels from being interpreted as characters.

5. Illumination and Contrast Enhancement

Contrast is improved using histogram equalization.

```
gray = cv2.equalizeHist(gray)
```

For documents with uneven illumination, CLAHE can provide better local contrast.

```
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced = clahe.apply(gray)
```

This makes characters more visible in both dark and bright regions.

6. Image Segmentation

Thresholding separates the text from the document background.

```
_, binary = cv2.threshold(
    enhanced,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Otsu's thresholding automatically selects a suitable threshold value.

For documents with uneven lighting, adaptive thresholding can be used:

```
binary = cv2.adaptiveThreshold(
    enhanced,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    31,
    10
)
```

This allows text to be separated from backgrounds with varying brightness.

7. Morphological Processing

Morphological operations are used to remove small unwanted regions and improve character shapes.

```
kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT,
    (2, 2)
)

binary = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel
)
```

Opening removes small noise while preserving the important text regions.

Morphological closing can also be used to connect broken character components.

```
binary = cv2.morphologyEx(
    binary,
    cv2.MORPH_CLOSE,
    kernel
)
```

8. Text Region Detection

Contours can be used to identify potential text regions.

```
contours, _ = cv2.findContours(
```

```
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

Bounding boxes can be generated around detected regions.

```
for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)

    if w > 10 and h > 10:
        cv2.rectangle(
            image,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )
```

This helps isolate text from unnecessary regions such as borders and images.

9. OCR Recognition

After preprocessing, the cleaned image is passed to an OCR engine such as Tesseract.

```
import pytesseract

text = pytesseract.image_to_string(binary)

print(text)
```

OpenCV performs the image enhancement and segmentation, while the OCR engine recognizes the characters.

10. Post-Processing

The extracted text can be cleaned by removing unnecessary spaces and line breaks.

```
text = " ".join(text.split())
```

This improves the readability and consistency of the final output.

Batch Processing

For processing a large number of documents, images can be processed one by one from a folder.

```
import glob

files = glob.glob("documents/*.jpg")

for file in files:
    image = cv2.imread(file)

    # Apply preprocessing
    # Perform OCR

    print("Processed:", file)
```

Processing documents sequentially reduces memory consumption because the entire dataset does not need to be loaded into memory at once.

Complete Pipeline

Input Scanned Document

↓

Image Acquisition

↓

Image Resizing

↓

Grayscale Conversion

↓

Noise Removal

↓

Contrast Enhancement

↓

Illumination Correction

↓

Thresholding / Segmentation

↓

Morphological Processing

↓

Text Region Detection

↓

OCR Recognition

↓

Text Post-Processing

↓

Final Extracted Text

Handling the Given Constraints

Requirement	**Solution**
Low-resolution images	Image upscaling using interpolation
Noisy documents	Gaussian/median filtering and morphological operations
Varying fonts	OCR engine processes different character shapes
Varying illumination	CLAHE and adaptive thresholding
Text-background separation	Otsu or adaptive thresholding
Broken characters	Morphological closing
Large number of documents	Sequential batch processing
Memory efficiency	Process one document at a time
Recognition accuracy	Enhancement, segmentation and noise removal before OCR

Conclusion

The proposed OCR system uses OpenCV for image acquisition, resizing, grayscale conversion, noise removal, contrast enhancement, illumination correction, thresholding, morphological processing, and text-region detection. The processed image is then given to an OCR engine for text recognition. Upscaling improves low-resolution text, filtering removes scanning noise, CLAHE handles uneven illumination, and adaptive thresholding separates text from the background. Sequential batch processing keeps memory usage low and makes the system efficient for processing large collections of scanned documents.

Code of Conduct:

I G.Joel Richard (192524054) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understan ding	**25%**	**Clearly defines the problem and identifies all relevant constraints accurately**	**Identifies most constraints with minor gaps**	**Partial understandi ng; misses key constraints**	**Fails to identify constraints correctly**
Constraint Analysis	**25%**	**Analyzes constraints effectively and prioritizes them logically**	**Provides reasonable analysis with some prioritizatio n**	**Limited analysis; weak prioritizatio n**	**No meaningful analysis or prioritizatio n**
Solution Developme nt	**25%**	**Develops optimal and feasible solutions satisfying all constraints**	**Develops workable solutions with minor limitations**	**Solutions partially satisfy constraints**	**Solutions do not meet constraints**

Applicatio n of Concepts	**15%**	**Effectively applies appropriate concepts, methods, or tools**	**Applies concepts with minor errors**	**Limited application; requires guidance**	**Unable to apply relevant concepts**
Justificatio n	**10%**	**Provides strong justification for decisions with logical reasoning**	**Provides justification with some clarity**	**Weak or unclear justification**	**No justification provided**